

Небезопасная десериализация

Эта уязвимость возникает, когда данные из недоверенного источника используются для манипуляции логикой процесса десериализации, что позволяет атакующему выполнить код, инъекцию или каким-то другим способом манипулировать объектами языка.

Многие языки программирования позволяют выполнять сериализацию и десериализацию нативных объектов. В некоторых случаях этим механизмом можно воспользоваться для выполнения атак.

Python

[pickle](#)

Описание проблемы

Это модуль из стандартной библиотеки Python для сериализации объектов. Он небезопасен "by design", т.к. при десериализации объектов возможно выполнение произвольного кода.

Последствия эксплуатации

Выполнение произвольного кода при десериализации объекта.

Пример эксплуатации уязвимости

```
1 import pickle
2 import os
3
4 class RCE:
5     # Метод __reduce__ предназначен для управления процессом
    десериализации.
6     # Он будет выполнен при десериализации объекта.
7     def __reduce__(self):
8         cmd = ("echo vulnerable")
9         return os.system, (cmd,)
10
11 serialized = pickle.dumps(RCE())
12
13 # При десериализации будет выполнена команда "echo vulnerable",
14 # т.е. на stdout будет выведена строка "vulnerable".
15 _ = pickle.loads(serialized)
16
```

Как исправить?

Важно, чтобы десериализация выполнялась только для тех данных, которым можно доверять. Ни в коем случае нельзя выполнять десериализацию данных, полученных от пользователя.

Пример безопасного использования

Если обрабатываются данные из недоверенного источника, избежать выполнения кода при десериализации нельзя. В этом случае модуль pickle использовать нельзя.

[dill](#)

Описание проблемы

Модуль расширяет возможности pickle и позволяет работать с большим набором типов объектов Python.

Последствия эксплуатации

Выполнение произвольного кода при десериализации объекта.

Пример эксплуатации уязвимости такой же, как и для pickle. Отличается только название модуля:

```
1 import dill
2 import os
3
4 class RCE:
5     def __reduce__(self):
6         cmd = ("echo vulnerable")
7         return os.system, (cmd,)
8
9 serialized = dill.dumps(RCE())
10
11 # При десериализации выполнится системная команда, указанная в
12 # методе __reduce__ сериализованного класса.
13 dill.loads(serialized)
```

Как исправить?

Как и в случае pickle, можно выполнять десериализацию только тех данных, которым можно доверять, нельзя выполнять десериализацию данных, полученных от пользователя.

Пример безопасного использования

Как и pickle, модуль dill нельзя использовать для десериализации данных, полученных из недоверенного источника.

PyYAML

Описание проблемы

Модуль часто используется для чтения файлов конфигурации, однако его возможности практически также широки, как и у pickle. Загрузка yaml, переданного из недоверенного источника, может привести к выполнению кода при его разборе.

- В старых версиях PyYAML < 5.1 при вызове метода "yaml.load" по умолчанию использовался небезопасный парсер.
- Начиная с версии PyYAML 5.1 по умолчанию стал использоваться безопасный парсер. Однако в нем несколько раз находили проблемы, позволяющие обойти добавленные ограничения: [\[1\]](#), [\[2\]](#), [\[3\]](#).
- Актуальная на текущий момент версия PyYAML 6.0.1 не содержит известных уязвимостей и не приводит к выполнению кода **при использовании безопасного парсера**. Также она требует обязательного указания парсера при использовании метода yaml.load (т.е. больше нет парсера "по умолчанию").

Последствия эксплуатации

Выполнение произвольного кода при десериализации объекта.

Пример эксплуатации

```
1 import yaml
2 import os
3
4 class RCE:
5     def __reduce__(self):
6         cmd = ("echo vulnerable")
7         return os.system, (cmd,)
8
9 serialized = yaml.dump(RCE())
10
11 # В переменной serialized будет примерно такая строка:
12 # '!!python/object/apply:posix.system\n- echo vulnerable\n'
13
14 # Т.к. используется небезопасный парсер yaml.Loader, выполнится
```

```
системная команда
15 # "echo vulnerable".
16 yaml.load(serialized, Loader=yaml.Loader)
17
```

Как исправить?

- Использовать актуальную версию PyYAML. На данный момент это версии 6.0.1.
- При обработке данных из недоверенного источника использовать безопасный парсер:

```
1 parsed = yaml.load(data, Loader=yaml.SafeLoader)
2 parsed = yaml.safe_load(data)
```

- Остальные парсеры можно использовать только если вы абсолютно уверены, что данные приходят из доверенного источника и пользователи не могут повлиять на их содержимое.

Пример безопасного использования

```
1 import yaml
2
3 # Например, злоумышленник передает на вход данные с целью выполнить
  команду системы.
4 # Безопасный парсер при загрузке таких данных выбросит исключение.
5 data = b"""
6 !!python/object/apply:posix.system ["echo vulnerable"]
7 """
8
9 # Для использования безопасного парсера можно указать его явно в
  функции
10 # yaml.load.
11 parsed = yaml.load(data, Loader=yaml.SafeLoader)
12
13 # Или использовать функцию yaml.safe_load:
14 parsed = yaml.safe_load(data)
```

[ruamel.yaml](#)

Описание проблемы

Это альтернатива PyYAML, поддерживающая стандарт YAML 1.2.

Последствия эксплуатации

Выполнение произвольного кода при десериализации объекта.

Пример эксплуатации

```
1 from ruamel.yaml import YAML
2
3 data = b"""
4 !!python/object/apply:posix.system ["echo vulnerable"]
5 """
6
7 # "unsafe" - на данный момент единственный встроенный парсер модуля
  ruamel.yaml,
8 # поддерживающий десериализацию объектов Python.
9 yaml = YAML(typ="unsafe")
10
11 # При загрузке будет выполнена системная команда "echo vulnerable",
  т.к.
```

```
12 # используется небезопасный парсер.
13 yaml.load(data)
14
```

Как исправить?

Как и в случае PyYAML: использовать безопасный парсер для загрузки данных из недоверенных источников. В ruamel.yaml безопасный парсер используется по умолчанию.

Примеры безопасного использования

```
1 from ruamel.yaml import YAML
2
3 data = b"""
4 !!python/object/apply:posix.system ["echo vulnerable"]
5 """
6
7 # По умолчанию используется парсер "round-trip", это эквивалентно
8 # значению typ="rt".
9 yaml_rt = YAML()
10
11 # При использовании парсера round-trip загрузка не вызовет исключения,
12 # однако класс Python не десериализуется.
13 # В данном случае функция вернет список ['echo vulnerable']
14 yaml_rt.load(data)
15
16 # Также можно указать безопасный парсер:
17 yaml_safe = YAML(typ="safe")
18
19 # В этом случае загрузка сериализованного класса Python вызовет
20 # исключение.
21 yaml_safe.load(data)
```

jsonpickle

Описание проблемы

Это модуль, позволяющий выполнять сериализацию и десериализацию сложных объектов Python в JSON и обратно. Как и модуль pickle, он небезопасен by design.

Последствия эксплуатации

Выполнение произвольного кода при десериализации объекта.

Пример эксплуатации

```
1 import jsonpickle
2
3 # jsonpickle сериализует объекты Python в JSON примерно такого вида
4 # (в данном случае это экземпляр класса RCE из примера для модуля
5 # pickle)
6 serialized = """
7 {
8   "py/reduce": [
9     { "py/function": "posix.system" },
10    { "py/tuple": [ "echo vulnerable" ] }
11  ]
12 }
13 """
```

```
14 | # При выполнении загрузки сериализованных данных будет выполнена
    | системная команда
15 | # "echo vulnerable".
16 | _ = jsonpickle.decode(serialized)
```

Как исправить?

Не выполнять десериализацию данных, полученных от пользователя. Модуль можно использовать только для обработки данных, полученных из доверенного источника.

Пример безопасного использования

Как и `pickle`, модуль `jsonpickle` нельзя использовать для десериализации данных, полученных из недоверенного источника.

Node.js

Десериализация при помощи `eval`

Описание проблемы

Функция `eval` не предназначена для десериализации, однако разработчик может исхитриться и все-таки попытаться использовать её для этих целей:

```
1 | const data = `{"hello": "there"}`
2 | const result = eval(`(${data})`)
```

Разумеется, если вместо JSON на вход этой функции передать какое-то произвольное выражение на JS, оно будет выполнено.

Также, например, библиотека [serialize-javascript](#) не имеет встроенного метода десериализации и предлагает для этих целей использовать `eval`.

Последствия эксплуатации

Выполнение произвольного кода при десериализации объекта.

Как исправить

Не использовать функцию `eval` для десериализации данных, полученных из недоверенного источника. В том числе - для загрузки JSON. И вообще - не использовать функцию `eval`.

Для загрузки JSON нужно использовать специально предназначенную для этой цели функцию `JSON.parse`:

```
1 | const data = `{"General": "Kenobi"}`
2 | const result = JSON.parse(data)
```

Пример безопасного использования

Функцию `eval` нельзя использовать для десериализации данных, полученных из недоверенного источника.